

Cosmos+ OpenSSD Project#3

Key-Value Interface 개통 및
Key-Value SSD 펌웨어 구현 실습

임베디드시스템소프트웨어

담당 교수: 김영재

이름 / 학번: 한유승 / 20211607

수행 기간: 25.05.28 ~ 25.06.11

1. 프로젝트 개요 및 목표

- 본 프로젝트의 목표와 Task 흐름에 대해 서술하시오.

기존 Cosmos + OpenSSD GreedyFTL 펌웨어의 NVMe I/O는 Logical Block (혹은 slice) 기반 I/O 인터페이스만을 제공하고, Key - Value SSD 인터페이스를 제공하지 않는다.

본 프로젝트에서는 host가 직접 정의한 key를 기준으로 value를 저장하고 검색하는 KV-SSD를 구현한다. 이를 위한 기능적 목표는 다음과 같다.

- KV-PUT command를 해석하여 value를 SSD 내부 logical address space에 저장한다.
- KV-GET command를 해석하여 해당 key의 최신 value를 반환한다.
- 동일 key에 여러 번 PUT이 발생하면 가장 최신의 write를 반영한다.
- 존재하지 않는 key에 대한 GET은 host가 인식 가능한 No such key 상태로 완료한다.
- 기존 GreedyFTL의 LSA-VSA변환, DMA request pipeline 등을 모두 지원할 수 있게 기존 구조와 유사한 흐름으로 작성한다.

2. 프로젝트 내용) Task1 – Key-Value Interface 개통 및 KV-SSD 펌웨어 개발

- A. 새로 정의한 NVMe KV I/O Command와 기존 NVMe Block I/O Command 간의 차이, 그리고 이러한 Command 변형을 통한 새 Storage Abstraction (Interface) 정의가 호스트 소프트웨어 스택에 미칠 수 있는 영향 서술

➤ ... (차이를 설명하는 시각 자료 필수 포함)

전체적으로는 Key value FTL과 Block(object) FTL 다음과 같은 차이가 있다.

Key Value	Object Storage
Data stored based on a key on native KV device	Data stored based on an object identifier on Block storage device
Key is variable length	Object Identifier is fixed length
Storage provides mapping of Key to Value	Protocol provides mapping of Identifier to Object
Key-Value storage is device level only	Object Storage may be split across multiple levels
No Metadata associated with value	Metadata is associated with object
Value is complete on the Key-Value device	Object may be split across devices

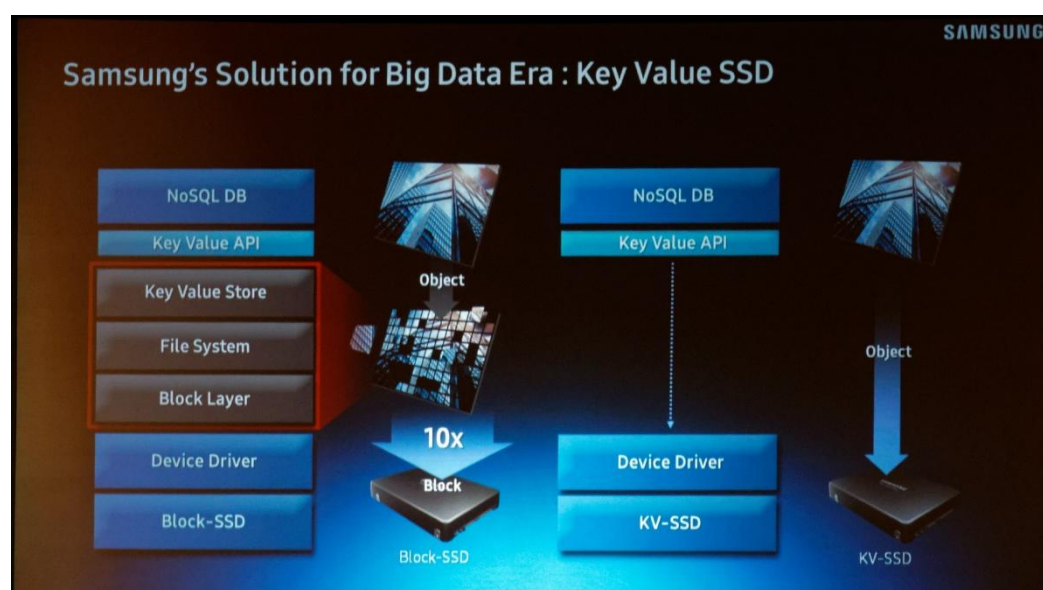
기존 block I/O command는 host가 데이터의 위치(LSA, LBA)를 직접 지정하는 반면, KV I/O command는 위치 대신 key를 전달하며, 데이터가 저장되는 위치의 결정은 전부 SSD 펌웨어 내부에서 결정된다. 두 커맨드의 SQ Entry 필드를 비교하면 다음과 같다.

구조체 필드	Block Write/Read(0x01/0x02)	KV PUT/KV GET(0xA0/0xA1)
OPC	0x01, 0x02	0xA0, 0xA1
CDW10-11	starting LBA (32 + 32 bit)	4byte Key
CDW12	NLB	NLB
CDW13		value length (bytes단위)
PRP	data buffer	PUT value, GET 결과 buffer
DW0(specific, completion)	X	GET시 실제 value length를 반환!

빨간색으로 표시한 필드가 주요 변경점이다.

- 기존에는 LBA의 시작 위치가 적혀있었는데, 이제 address대신 KEY가 들어간다. 따라서 host는 name -> location translation이 불가능해진다.
- Completion (DW0)의 변화: Block read는 host가 요청한 길이를 그대로 돌려받으므로 value length를 확인할 필요가 없다. 하지만 KV -GET은 host가 value의 실제 길이를 모르는 상태로 요청하므로, device가 completion DW0(specific필드)에 value 길이를 실어 반환해야 한다.
또한 존재하지 않는 key에 대해서는 CQ entry에 SCT= 0x7, SC=0xC1을 넣어서 ioctl 반환 status 0x7C1이 되어 kvbench에서 no such key로 처리된다.

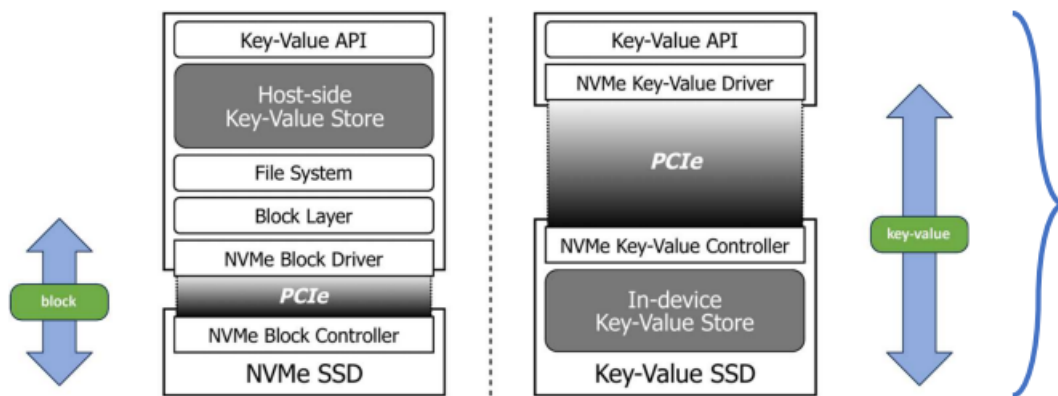
주소 변환 Flow는 다음과 같이 변화한다.



즉 block I/O는 App-> FS -> Block layer -> nvme driver -> LBA(host) -> LSA(NAND) -> VSA -> physical NAND 로 많은 layer를 거쳐야 한다.

하지만 KV에서는 APP -> KV API -> key -> SSD value (LBA -> LSA -> physical NAND) 로 host에게 상당부분 책임을 넘긴다.

block level FTL -> KV SSD로의 변화는 host 소프트웨어 스택에 다음과 같은 영향을 끼친다.



Comparison of Software Stacks for the Conventional Block Interface and the New Key-Value Interface

- file system 단에서 KV메소드를 구현하면 key - file offset - LBA 변환을 모두 host 에서 수행했어야 했다. KV SSD에서는 해당 단계가 device로 내려가므로, 해당 layer를 생략하여 host에서 CPU 자원 오버헤드를 줄이고, 구현의 편의성을 얻을 수 있다.
- 기존 block level SSD에서 KV를 구현하면, 같은 KEY에 대해 최신 value를 유지할 책임이 host에 있었다. KV SSD를 사용하면, 해당 관리를 SSD내에서 하기 때문에, value의 신뢰성이 올라간다.

B. KV-SSD 구현체의 Index 자료구조에 대한 설명

➤ ... (채택한 Index 구조에 대한 그림 + 선정 이유 + 장단점을 모두 서술)

KV index는 kv_store.h에 hash table로 선언하였고, 이를 memory_map.h에서 device DRAM의 reserved 영역에 배치하였다.

```
#define KV_INDEX_ENTRY_COUNT      (1 << 23) // 8,388,608 entries
#define KV_INDEX_ENTRY_MASK      (KV_INDEX_ENTRY_COUNT - 1)
```

```
typedef struct _KV_INDEX_ENTRY {
    unsigned int key;
    unsigned int startLba;
    unsigned int valueLength;
} KV_INDEX_ENTRY, *P_KV_INDEX_ENTRY;
```

hash는 충돌 시 다음 slot으로 한 칸씩 이동하고, 빈 entry는 startLba를 KV_LBA_NONE으로 표시하게 초기화 하였다.

선택 이유와 장단점은 다음과 같다.

- hash 자료구조는 평균적으로 $O(1)$ 의 상수 시간 lookup/insertion을 제공하여 서로 다른 키를 통해 많은 양의 request가 들어오는 kv bench에 장점이 있다.
- 원래는 key가 가변적이지만, 여기에서는 key가 고정크기이기 때문에, 가변길이 key를 지원할 필요가 없어 단순한 entry를 사용하였다.
- 단, DRAM에만 해당 entry가 존재하여 persistency가 없다.
- LSM tree와 달리, range query, 순회 등의 정렬기반 연산을 지원할 수 없다는 본질적 한계가 있다.

C. KV-SSD 구현체의 Value 관리 기법에 대한 설명

➤ ... (특정 Key에 대한 최신 Value를 어떻게 저장, 검색하는지 서술)

저장: KV_PUT

KB PUT은 nvme_io_cmd.c의 handle_nvme_kv_put()에서 처리하며, NVMe block 단위(4KB) 할당과 동일 key overwrite시에 기존 LBA를 재활용하게 하여 대용량 데이터를 처리할 수 있게 하였다. 처리 흐름은 다음과 같다.

1. CDW10에서 key, CDW12에서 NLB, CDW13에서 value 길이 추출
2. KvFtlPut() 호출
 - A. 기존 entry가 있고 새 value가 기존 block 수 이하로 들어가면
-> 기존 startLba를 그대로 재사용 (in-place overwrite)
 - B. 그 외에는 전역 변수 nextKvLba로 새 LBA 구간 할당
 - C. hash index에 key -> (startLba, valueLength) 갱신
3. ReqTransNvmeToSlice()로 기존 GreedyFTL write pipeline에 전달
-> slice request 생성 -> data buffer 할당 -> RX DMA로 host value 수신

-> LSA→VSA 변환 -> NAND program

10,000,000 테스트 이전에는 value 하나에 slice(16 KiB) 전체를 할당했기 때문에, benchmark의 4 KiB value 기준으로 PUT마다 12 KiB의 logical space 낭비가 있었다.

테스트 실패 후, 이를 NVMe block(4 KiB) 단위 할당으로 수정하여 낭비를 제거하였다. 또한 초기에는 동일 key 재-PUT 시에도 매번 새 LBA를 append하여 logical space가 단조 증가했는데, 새 value가 기존 자리에 들어가는 경우 기존 LBA 구간을 재사용하도록 개선하였다. 이 두 가지 개선이 없으면 10,000,000회 PUT 시 $10M \times 16 \text{ KB} = 152.6 \text{ GB}$ 의 logical space가 필요해 용량을 초과하지만, 개선 후에는 unique key 수만큼만(약 $3.8M \times 4 \text{ KB} = 14.5 \text{ GB}$) 소비된다.

검색: KV_GET

KV GET은 handle_nvme_kv_get()에서 처리한다. 처리 흐름은 다음과 같다.

1. CDW10에서 key 추출
2. FindKvIndexEntry()로 hash lookup
 - A. 실패 시: NAND 접근 없이 status에 0x7c1로 CQ반환
-> host는 ENOSUCHKEY로 해석
3. 성공 시 : ReqTransKvGetToSlice(startLba, readNlb, valueLength) 호출
-> 기존 read pipeline 재사용 (LSA->VSA 변환, NAND read, TX DMA)
4. TX DMA완료시 수동 completion으로 CQ DW0에 valueLength 기록

GET 경로에서 핵심적인 변경은 completion 처리다. 기존 block I/O는 DMA 발행 시 hardware auto-completion(HOST_DMA_CMD_FIFO_REG의 autoCompletion bit)을 켜서 DMA 완료와 동시에 hardware가 CQ entry를 자동 생성한다. 그러나 이 방식으로는 completion DW0에 value 길이를 실을 수 없다. 따라서 KV-GET request는 NVME_DMA_INFO에 추가한 autoCompletion/completionSpecific 필드를 통해 auto-completion을 끄고 value 길이를 request metadata에 보관하며, CheckDoneNvmeDmaReq()에서 TX DMA 완료를 확인한 뒤 set_auto_nvme_cpl(cmdSlotTag, valueLength, 0)로 수동 completion을 발행한다.

즉 KV index의 value length는

1. reqPool.nvmeDmaInfo.completionSpecific에 저장
2. TX DMA완료시 CheckDoneNvmeDmaReq()에서 읽음
3. set_auto_nvme_cpl()로 CQ entry specific DW0에 기록
4. host의 ioctl cmd.result로 전달.

D. 10,000,000번 PUT (Keyspace 4,194,304) 테스트 시 결과 캡처 & 분석

- ... (kv_bench가 FAIL 없이 성공한 경우
→ 교수님과 논의 후 최종 프로젝트 점수에 일부 가산점 부여)

```
embe2024@embe2024-PNVKB0A0-Samsung-Desktop:~/workspace/prj3-materials/prj3-host-evaluation$ sudo ./kv_bench /dev/nvme0n1 10000000 4194304 1
[sudo] embe2024 암호:

[INFO] GET failed @ key `{` (No such key)
-----
Cosmos+ OpenSSD-Based KV-SSD Benchmark
-----
ops=10000000 keyspace=4194304
unique_keys=3807651
result: OK=3807651 FAIL=0 NO-SUCH-KEY=1
elapsed: 7.70468e+06 ms (2595.82 IOPS est. for PUT+GET )
-----
```

- put: 같은 key에 대한 overwrite를 포함한 10,000,000개 모두 성공하였다.
- get: put에 대한 read모두 성공하여 FAIL=0을 달성했다.
- no such key: invalid key 1개도 정상 처리하였다.
- 7705초 약 2시간 8분정도 소요되었으며, 2596 IOPS로 측정되었다.
- slice 단위로 append하던 해당 테스트 이전 초기 저장방식을 사용했으면, 약 150GB의 공간이 필요하였을 것이다. 하지만 LBA단위 저장과 overwrite시 LBA재활용을 통해 약 15GB까지 저장 용량을 줄여서 성공했다.
- 평가에 사용한 hshot 컴퓨터의 메모리가 작아, 기존 kv_bench를 그대로 실행했다가 OOM이 발생했다. 따라서 benchmark의 host 검증용 자료구조의 크기를 줄였다.

3. 참고 문헌

- 보고서 작성 간 최소 하나 이상의 공인 학술 문헌 (e.g., 논문, 기술 리포트, 기술 Specification 등)을 참조하시오.

Seok, K. Y. (2017). Key value SSD explained: Concept, device, system, and standard. Storage Networking Industry Association (SNIA).

수업 강의자료: 13chapter

FMS 2017: Samsung Announces QLC V-NAND, 16TB NGSFF SSD, Z-SSD V2, Key Value presentation.